3번 문제 설명 및 코드 설명

- **문제 3**

3-1. 프로세서가 처음 동작할 때 모터 엔코더를 돌려 원하는 비밀번호 정하고 P1.1 스위치를 눌러 저장

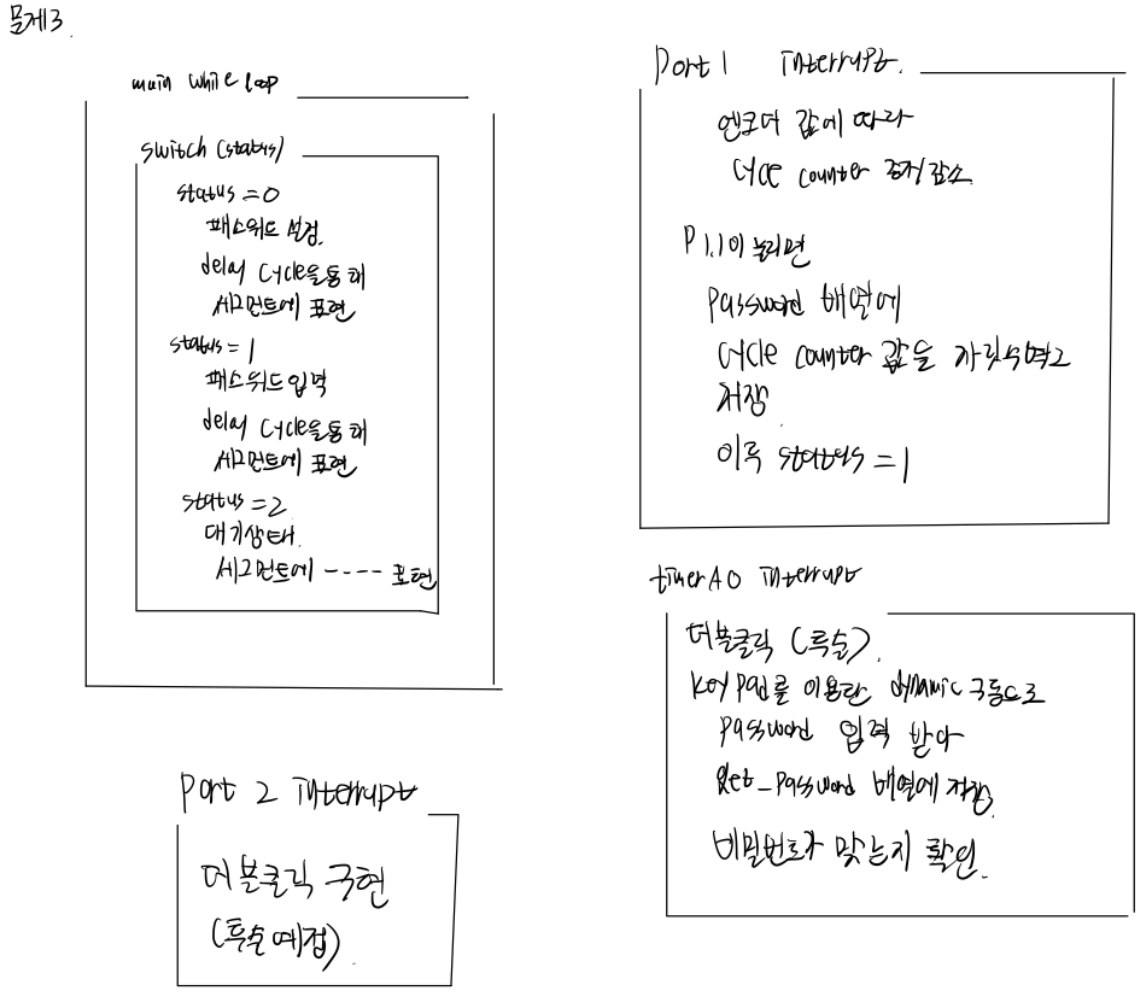
3-2. 키패드는 한자리 숫자를 입력하고 별(*)을 눌러서 4자리 숫자를 만들게 하고, 마지막 별(*)을 눌러서 비밀번호가 맞으면 대기상태 진입

3-3. 비밀번호 입력중 샵(#)을 누르면 비밀번호 입력을 다시 하게 만들고, 비밀번호가 맞지 않는 경우에도 처음부터 비밀번호를 입력

3-4. 대기 상태에서 P2.1스위치 두 번 연속(둘 사이의 시간 0.3s이내)으로 누르면 잠금모드 3-2부터 실행 (추가 점수4)
스위치를 한 번 눌렀을 때는 P1.0 led가 점등이 되는 코드를 작성하여 클릭과 더블 클릭을 구분할 수 있는지를 확인하십시오.

코드 설명

- 코드 구조


문제3
main while loop
switch (status)
status = 0
패스워드 설정
delay 다이얼을 통해
세그먼트에 표현
status = 1
패스워드 입력
delay 다이얼을 통해
세그먼트에 표현
status = 2
대기상태
세그먼트에 ---- 표현
Port 1 Interrupt
엔코더 값에 따라
Cycle counter 증가/감소
P1.1이 눌리면
password 배열에
Cycle counter 값을 가지고와서
저장
이후 status = 1
Port 2 Interrupt
더블클릭 구현
(특수 예정)
timerA0 Interrupt
더블클릭 (특수)
key pad를 이용한 dynamic 구동으로
password 입력 받아
Ret_password 배열에 저장
비밀번호가 맞는지 확인

변수 설명

```
volatile int timer_gl = 0;
```

- 글로벌 타이머. 더블클릭 구현에 이용

```
volatile int cycle_counter = 5000;
```

- 초기 비밀번호 설정을 위한 엔코더값.

```
unsigned int number[10] = { 0xDB, 0x50, 0x1F, 0x5D, 0xD4,
0xCD, 0xCF, 0xD8, 0xDF, 0xDD };
```

- 세그먼트 0~9 표현 배열

```
unsigned int password[4] = {10, 10, 10, 10}; // initial value
> 9999 to prevent error
```

- 비밀번호가 저장될 배열. 각 자릿수 값은 0~9 이므로 초기 오류를 방지하기 위해 초기값 10으로 설정

```
unsigned int get_password[4]= {0};
```

- 입력받은 비밀번호를 저장하는 배열.

```
volatile unsigned int status = 0;
```

- 현재 상태(state)를 제어하는 변수

```
unsigned int numpad_input = 0;
```

- 각 자릿수 별로 입력받기 위한 카운트 변수

```
int i = 0;
```

- 비밀번호가 맞는지 확인하는 함수의 for문에서 사용할 변수.

```
volatile unsigned int button_pressed = 0;
```

- 더블클릭에서 사용할 첫 번째 클릭이 수행되었냐를 나타내는 변수

```
unsigned int timer_chattering = 0; // 1000 * 50
```

- 타이머를 이용한 채터링 제거에 사용할 변수

문제를 해결한 주요 Logic

1. 타이머를 이용한 채터링 제거

1.1) 앞서 2 번 문제에서와는 다른 방식으로 구현. 버튼을 누르면 timer_chattering = 50 의 값을 넣고, 1000us * 50 즉 50ms 마다 초기화.

버튼이 눌리면 먼저 버튼 인터럽트를 비활성화 한 후, timer interrupt 에서 timer_chattering 1씩 감소시킨다. 이후 timer_chattering이 0이 되면 다시 버튼 인터럽트를 활성화시킨다.

2. 패스워드 입력 logic

2.1) Numpad_input 변수는 get_password 배열의 자릿수를 뜻한다. *가 눌릴 때 마다 1 씩 증가시킨다. 즉 현재 활성화된 숫자가 get_password[numpad_input] 에 저장된다.

2.2) #을 누르면 numpad_input 변수와 get_password 배열을 0 으로 초기화한다.

2.3) Numpad_input = 4, 즉 마지막 자리인 get_password[3]에 값을 넣은 이후에 *를 누르면 numpad_input 변수와 get_password 배열을 0 으로 초기화한다.

3. 패스워드가 맞는지 확인하는 logic

3.1) 간단한 c 언어 함수와 for 문을 이용하여 맞으면 1, 틀리면 0 을 return 하는 함수를 만들었다.

3.2) Get_password 배열과 password 배열의 원소를 하나씩 확인하여 모두 맞으면 1, 하나라도 틀리면 0 을 return 한다.

4. 더블클릭 logic

4.1) 먼저 첫 번째 클릭을 수행하면 button_pressed 변수를 1 로, timer_gl 을 0 으로 초기화한다. 추가 문제 구현을 위해 p1.0 LED 도 이때 켜준다.

4.2) 이후 timer_gl 이 300 을 넘게 되면, 즉 1ms * 300 = 0.3sec 을 초과하게 되면 button_pressed 변수를 0 으로 초기화시킨다. 또한 p1.0 LED 를 꺼준다.

4.3) 0.3 초가 지나 변수를 초기화하기 전 까지 button_pressed 변수는 1 이므로 if 문을 활용하여 button_pressed = 1 일때 버튼 클릭이 감지되면, 즉 첫 번째 클릭과 두 번째 클릭의 시간 간격이 0.3 초 이내이므로 더블클릭이 구현된다.